

AWS Agreement RAG Chatbot

Project Overview

This project is a Retrieval-Augmented Generation (RAG) application built using FastAPI, Streamlit, ChromaDB, HuggingFace Embeddings, Gemini LLM, and SQLite.

The system allows users to ask questions about the AWS Customer Agreement document and receive context-aware answers generated using a RAG pipeline.

Features

- PDF Document Ingestion
 - Text Chunking using RecursiveCharacterTextSplitter
 - Local Embeddings using HuggingFace Sentence Transformers
 - Vector Storage using ChromaDB
 - Retrieval-Augmented Generation (RAG)
 - Gemini 2.5 Flash LLM Integration
 - FastAPI Backend
 - Streamlit Frontend
 - Query Logging using SQLite
 - Analytics Dashboard
 - Source Chunk Display for Transparency
-

Architecture

User Query ↓ Streamlit UI ↓ FastAPI API ↓ Retriever (ChromaDB) ↓ Relevant Chunks ↓ Gemini LLM ↓
Generated Answer ↓ Answer + Source Chunks

Tech Stack

Backend

- FastAPI
- LangChain

Frontend

- Streamlit

Vector Database

- ChromaDB

Embeddings

- sentence-transformers/all-MiniLM-L6-v2

LLM

- Gemini 2.5 Flash

Database

- SQLite
-

Project Structure

AWS_RAG/

├── main.py

├── ingest.py

├── rag_pipeline.py

├── database.py

├── streamlit_app.py

├── AWS Customer Agreement.pdf

├── requirements.txt

├── .env

├── agreements_db/

└── rag_logs.db

Installation

Clone Repository

```
git clone <repository_url>  
cd AWS_RAG
```

Create Virtual Environment

```
python -m venv ve
```

Activate Environment

Windows

```
ve\Scripts\activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Environment Variables

Create a `.env` file:

```
gemini=YOUR_GEMINI_API_KEY
```

Running the Application

Start FastAPI

```
uvicorn main:app --reload
```

API Documentation:

```
http://127.0.0.1:8000/docs
```

Start Streamlit

```
streamlit run streamlit_app.py
```

API Endpoints

POST /ingest

Loads the AWS Customer Agreement PDF, creates embeddings, and stores vectors in ChromaDB.

Response:

```
{
  "message": "PDF Ingested Successfully"
}
```

POST /ask

Ask questions related to the AWS Customer Agreement.

Request:

```
{
  "question": "What are End Users?"
}
```

Response:

```
{
  "answer": "...",
  "sources": [
    "Retrieved source chunk"
  ]
}
```

GET /analytics

Returns analytics data collected from user queries.

Response:

```
{
  "most_frequent_questions": [],
  "no_answer_queries": [],
  "average_latency": 0.85
}
```

Analytics Dashboard

The application tracks:

- Most Frequently Asked Questions
- Questions With No Answer
- Average Response Latency

These metrics are displayed through the Streamlit Analytics Dashboard.

Future Improvements

- Multi-document support
- User PDF Upload
- Conversation Memory
- Authentication
- Docker Deployment
- Cloud Deployment (AWS/Azure/GCP)

Author

Rithish Kumar Reddy

B.Tech – Computer Science and Engineering (Data Science)

AI / ML / Data Science Enthusiast